UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESTRUCTURA DE DATOS
CATEDRÁTICO: LUIS FERNANDO ESPINO BARRIOS
TUTOR ACADÉMICO: RUBÉN ALEJANDRO RALDA MEJÍA



MANUAL DE USUARIO
JAVALANG INTERPRETER

OTTONIEL FABRICIO VÁSQUEZ PINEDA
CARNÉ: 202307820
SECCIÓN: B

GUATEMALA, 17 DE SEPTIEMBRE DEL 2,025

1. Introducción

JavaLang Interpreter es una herramienta educativa diseñada para interpretar programas escritos en el lenguaje JavaLang, una variante académica con sintaxis similar a Java. Esta herramienta incluye una interfaz gráfica que permite editar código, ejecutarlo y visualizar reportes de análisis (AST, tabla de símbolos y errores).

2. Instalación

Requisitos del sistema:

- Sistema operativo Linux
- Compilador GCC
- Herramientas Flex y Bison
- Biblioteca GTK3
- Graphviz (para visualización del AST)

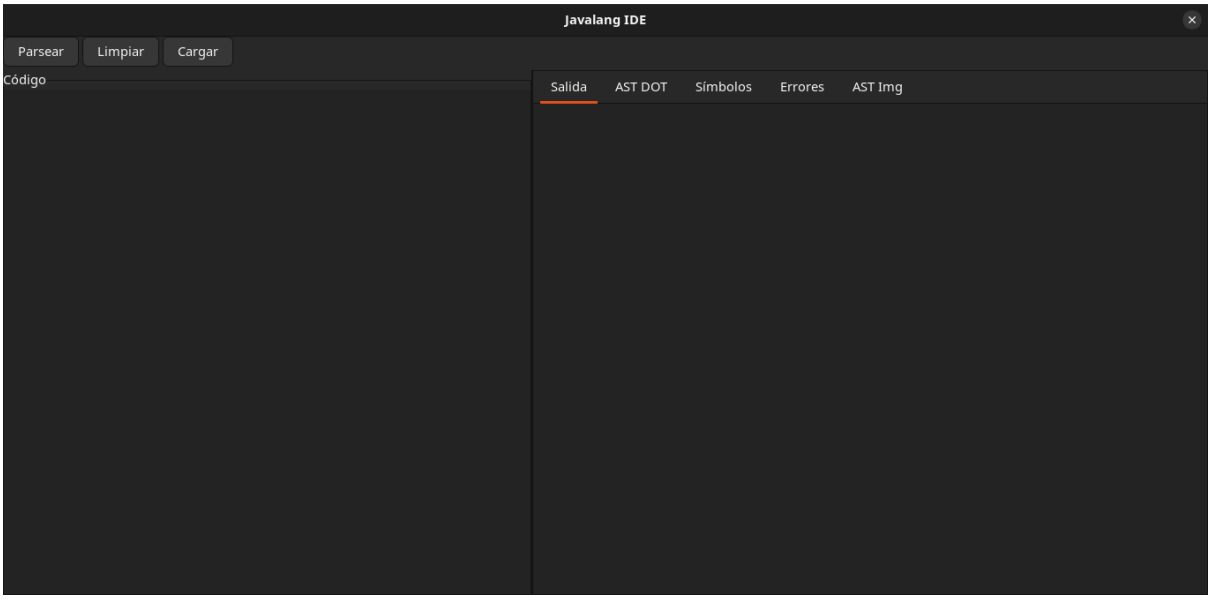
Pasos de instalación:

1. Actualizar lista de paquetes
   sudo apt update
2. instalación de herramientas
   sudo apt install flex bison

3. instalar GTK3 y las dependencias de desarrollo
   sudo apt install libgtj-3.dev

4. instalar Graphviz
   sudo apt install graphviz
5. Instalar herramientas adicionales
   sudo apt install git make cmake
6. Clonar el repositorio del proyecto:
   git clone https://github.com/Farot30/OLC2_2s2025_202307820
   cd OLC2_2S2025_202307820
7. Compilar proyecto
   make
8. Ejecutar la aplicación:
   ./run_gui.sh

3. Uso de la herramienta



La interfaz contiene las siguietnes áreas:

- Editor de código: para escribir código Javalang
- Panel de pestañas:
    - Salida de consola
    - Código DOT del AST
    - Tabla de símbolos
    - Reporte de errores
    - Visualización del AST como imagen.

Crear y Editar Código:

Haga clic en el editor de código.

Escriba su programa en lenguaje JavaLang

Ejecutar código:

1. Cargue un archivo existente con el botón “cargar” (archivo con extensión .usl).
2. Escriba código directamente en el editor.
3. Presione el botón “Parsear” para:
    - Analizar el código léxicamente.

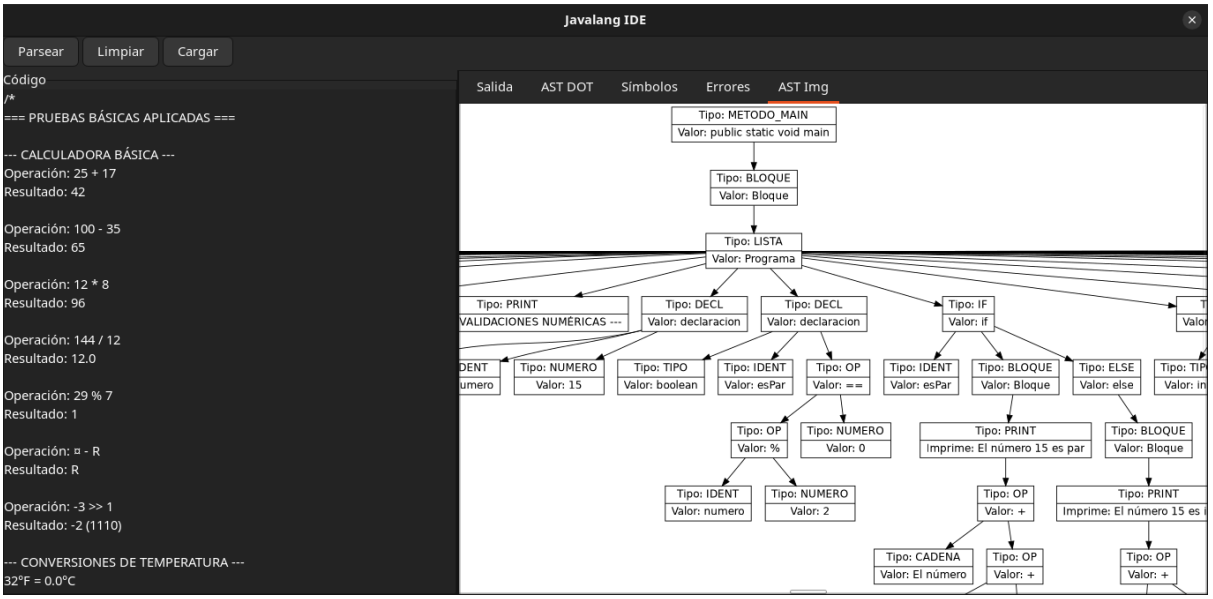
- Analizar sintácticamente.
- Realizar análisis semántico
- Generar todos los reportes

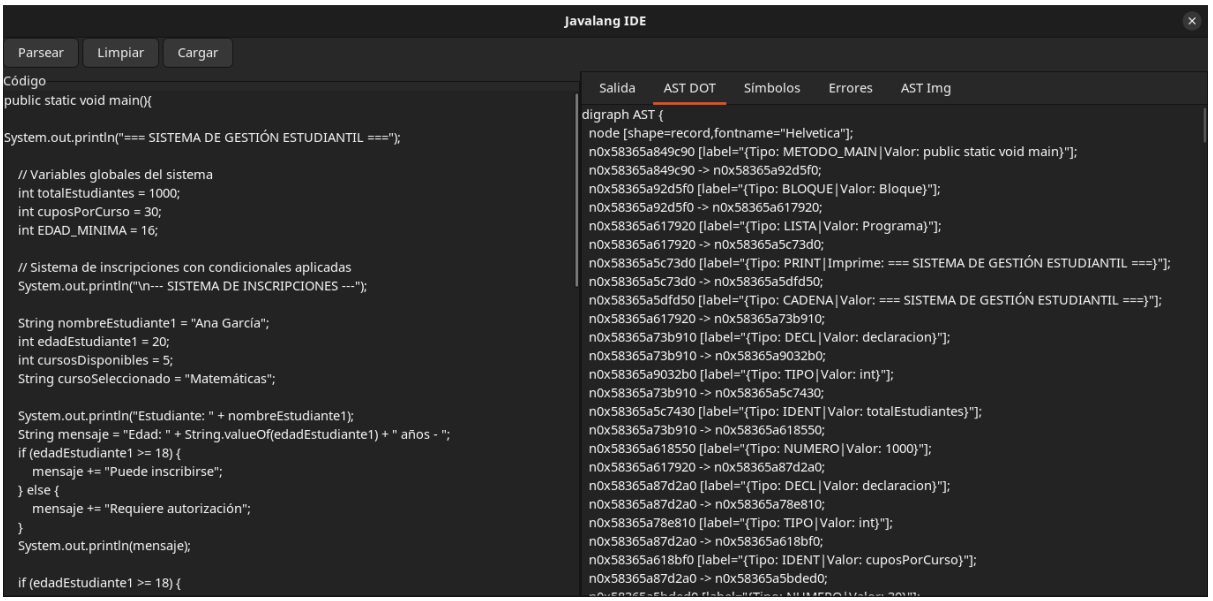
Ejemplo de código válido:

```
public static void main(){
    int x = 8;
    System.out.println(x);
}
```

Interpretación de Reportes

Reporte de AST:

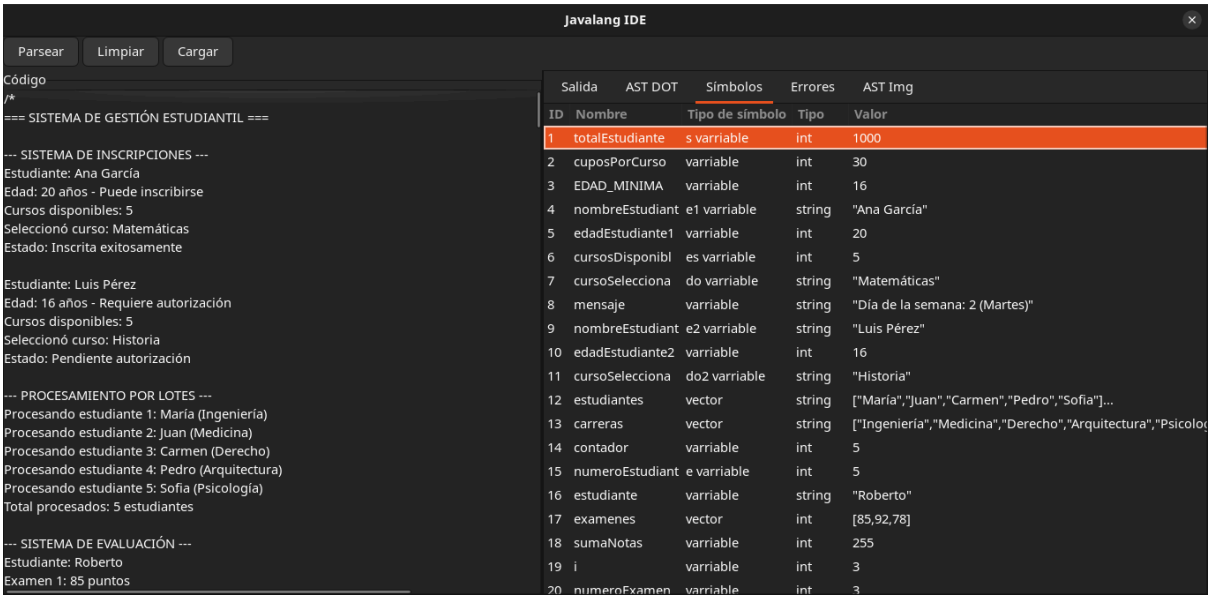




El AST (Abstract Syntax Tree) muestra la estructura jerárquica del código analizado. Se presenta en dos formatos:

1. Código Dot: Representación textual para expertos.
2. Imagen: Visualización gráfica generada con Graphviz

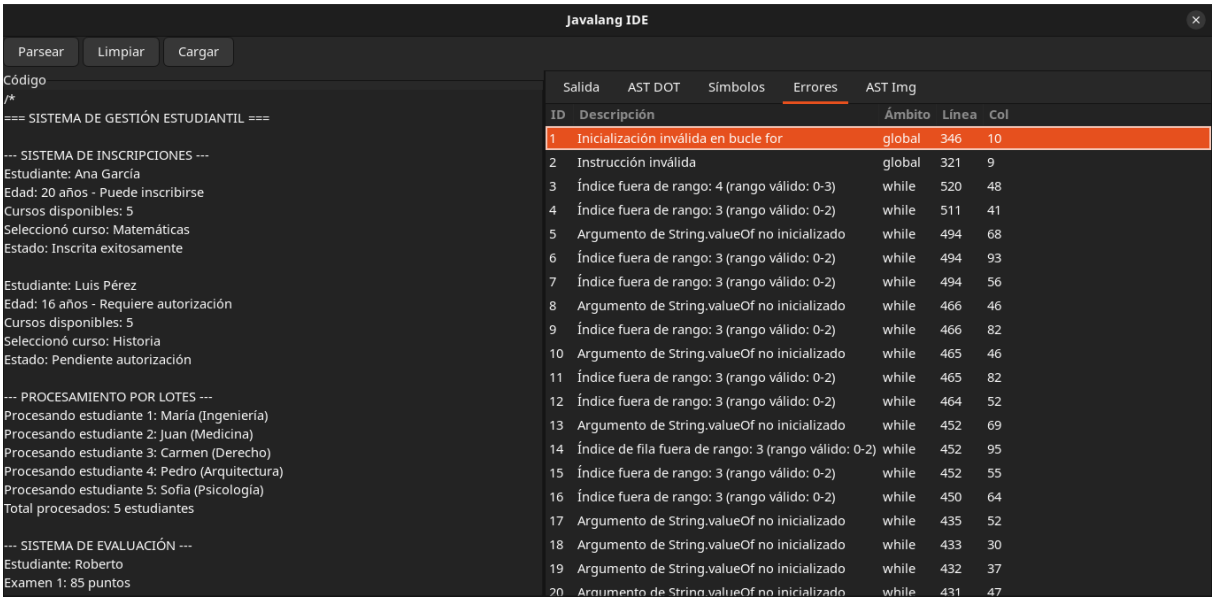
Tabla de Símbolos:



Muestra información detallada sobre:

- Variables declaradas (ID, nombre, tipo, valor, ámbito, línea, columna)
- Funciones definidas
- Tipos de datos utilizados
- Ámbitos de visibilidad

Reporte de Errores

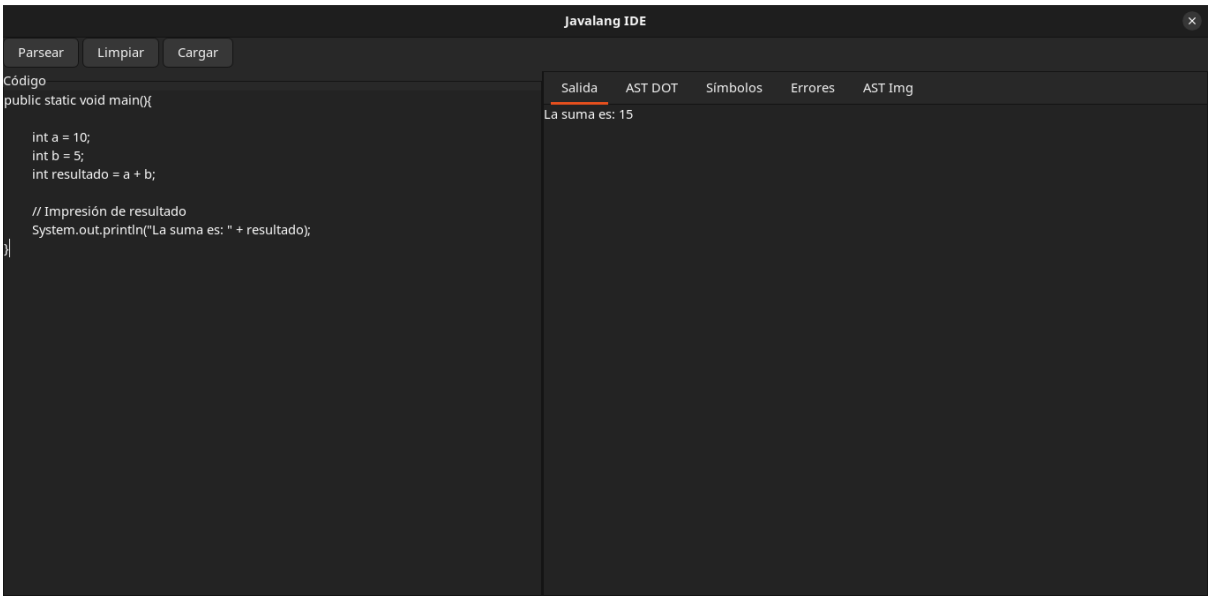


Clasifica los errores encontrados durante el análisis:

- Errores léxicos: Caracteres no válidos
- Errores sintácticos: Estructuras incorrectas
- Errores Semánticos: Tipos incompatibles, variables no declaradas

Ejemplos de Sesión de Uso

Ejemplo 1:

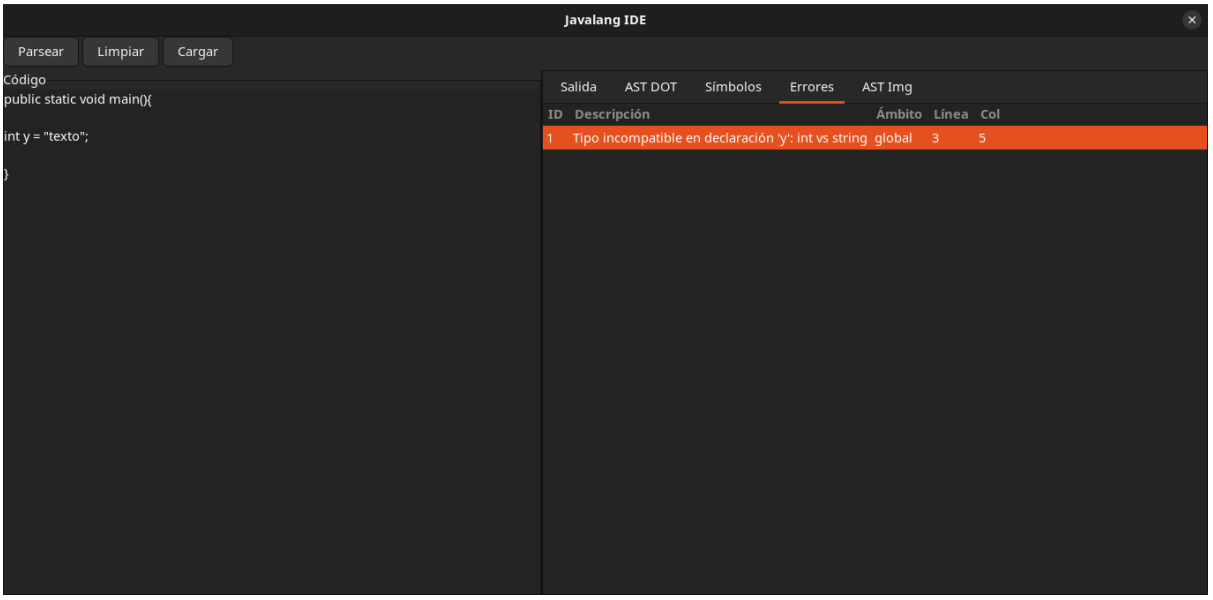



```
public static void main(){
        int a = 10;
        int b = 5;
        int resultado = a + b;
        // Impresión de resultado
        System.out.println("La suma es: " + resultado);
}
```

Resultado esperado:

- Salida en consola: "La suma es: 15"
- AST generado correctamente
- Tabla de símbolos con variables a, b, resultado
- Sin errores reportados

Ejemplo 2: Programa con Error semántico



Código de error

```
public static void main(){
        int y = "texto";
}
```

Resultado esperado

- Reporte de error semántico mostrado
- Mensaje: "No se puede asignar String a variable int"
- Tabla de símbolos muestra variable con error